

How to Maximize the Impact of Agentic AI in the SDLC

19 January 2026 - ID G00843405 - 19 min read

By: Manjunath Bhat, Philip Walsh, Bill Blosen, Mark O'Neill, Brian Minning

Initiatives: Use AI to Accelerate Productivity Throughout the SDLC; Delivery of Functional Responsibilities

Agentic AI reduces friction and boosts productivity only when diligently applied to multiple stages in the SDLC. Software engineering leaders can use this research to progressively resolve systemic bottlenecks and reimagine the SDLC by augmenting human experience and automating away inefficiencies.

Insights at a Glance

AI boosts productivity and reduces friction but only when diligently applied to multiple use cases and user roles in the SDLC. The 2025 Gartner AI in Software Engineering Survey shows that organizations implementing AI broadly throughout the SDLC are more likely to see improvements across most business outcomes. For example, 66% of respondents using AI for 10 (or more) use cases reported increased speed to market compared to 35% of respondents using AI for less than five use cases.

To attain the aspirational goal of an AI-native, zero-friction SDLC, software engineering leaders must adopt a three-phased approach:

Phase 1: Resolve Constraints by Mapping the Software Delivery Value Stream

- Identify systemic bottlenecks in the SDLC and resolve them by judiciously using AI and non-AI automation techniques
- Discover pain points and improve the experience for all roles, not just developers
- Establish internal developer platforms for consistency and scale

Phase 2: Reimagine the SDLC With Asynchronous Workflows

- Parallelize tasks using asynchronous agentic workflows

- Transform software delivery workflows — identify what steps can be eliminated and what stays the same
- Enhance IDP capabilities to govern, monitor and control AI software engineering agents

Phase 3: Realize Zero-Friction SDLC With Autonomous Software Delivery

- Implement autonomous self-improvement loops and address pitfalls
- Expand platform support for autonomous delivery and operations
- Implement appropriate human oversight for autonomous workflows based on business criticality, acceptable risk, and architectural complexity

Strategic Planning Assumptions

By 2028, asynchronous SWE agent workflows will improve software engineering team productivity by 30% to 50%, surpassing the 0% to 20% gains from AI code assistants in 2025.

By 2027, the number of **platform engineering** teams using AI to augment every phase of the SDLC will have increased from less than 5% to 40%.

By 2028, **AI agents** and assistants will remediate 70% of software code vulnerabilities, up from 10% today, by either updating dependencies or suggesting code fixes.

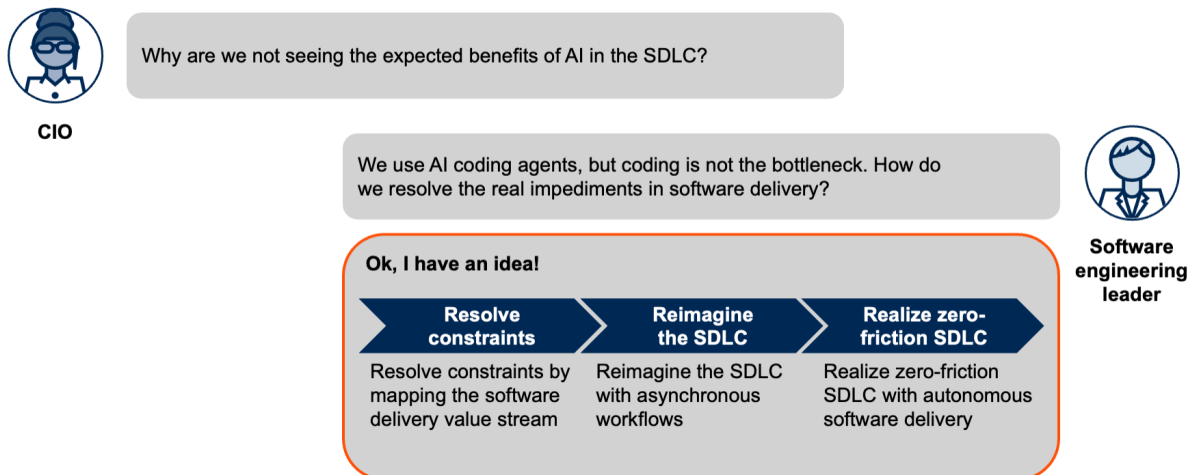
Why You Should Care

To maximize the impact of AI in the SDLC and realize return on investment (ROI), software engineering leaders must take a systematic approach. You need to aim for a “frictionless” flow of value across the entire life cycle by identifying and resolving the biggest constraints. This requires reimagining the SDLC using asynchronous and autonomous workflows supported by a developer platform with robust foundations. This concept is adapted from the book, *The Goal* in which Dr. Eliyahu Goldratt defines constraint as “what limits a system from achieving higher performance.”

Taking a structured and systematic approach to integrating AI into the SDLC helps software engineering leaders connect engineering improvements to business outcomes and meet C-level expectations of substantial productivity gains. See Figure 1.

Figure 1: How to Meet CIO Expectations of Substantial Gains in Developer Productivity

How to Meet CIO Expectations of Substantial Gains in Developer Productivity



Source: Gartner
843405

Gartner

Focusing solely on coding activities within the developer's IDE yields diminishing ROI beyond a certain low threshold — because the greatest constraint to value delivery can lie elsewhere. Effective adoption of AI tools and capabilities across a broader array of development activities leads to higher productivity gains. However, software engineering leaders can look beyond mere productivity gains to unlock greater business value. These value drivers include:

1. Greater competitive advantage by way of an increased number of innovative products and features
2. Increased user/customer satisfaction
3. Increased developer satisfaction leading to improved retention

Gartner's data bears this out. The 2025 Gartner AI in Software Engineering Survey shows a striking contrast between teams maximizing AI use across the SDLC versus those minimally using it for fewer use cases:

1. 55% of respondents who use AI for 10 or more use cases see an increase in the number of **innovative products and features**, compared to 38% who use AI for less than five use cases.

2. 53% of respondents who use AI for 10 or more use cases cite an increase in **user/customer satisfaction**, compared to 32% who use AI for less than five use cases.
3. 61% of respondents who use AI for 10 or more use cases report an increase in the **job satisfaction** of developers, compared to 44% who use AI for less than five use cases.

The teams that succeed with agentic development will be the ones who recognize that the entire software development life cycle needs to evolve in concert.

— *The New Calculus of AI-based Coding*, Joe Magerramov,
VP/Distinguished Engineer at AWS

Actions

Actions and Cautions

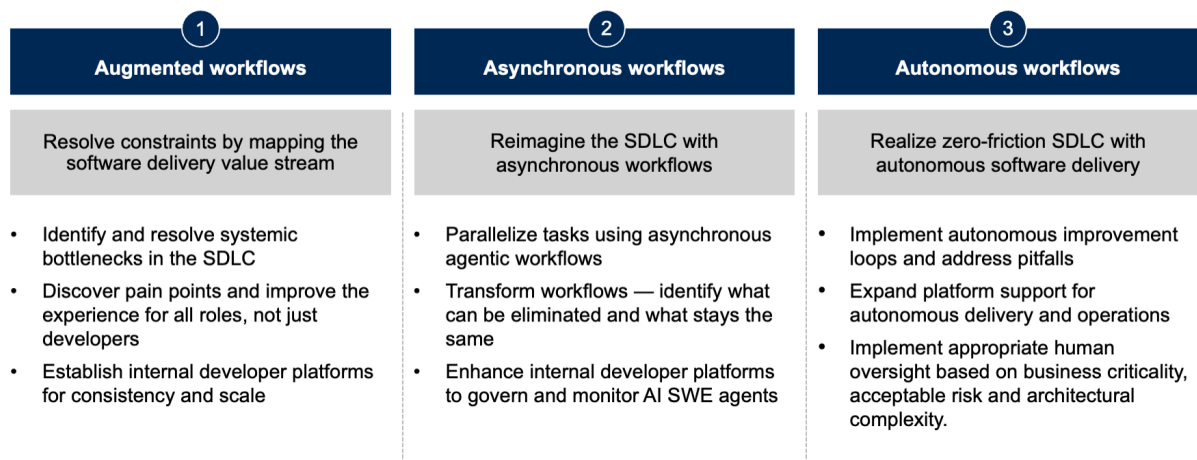
Software engineering leaders can realize the true value of **agentic AI** in the SDLC by moving beyond individual task efficiency and implementing a systematic, pan-SDLC strategy across multiple roles. This requires a phased approach starting with mapping the current software delivery value stream to identify critical bottlenecks. Then, deploying agentic AI to create autonomous SDLC workflows and finally using internal developer platforms to strengthen governance and mitigate the inherent risks of agentic systems.

Actions

Adopt a three-phased approach to steadily leverage and maximize the impact of agentic AI in the SDLC. See Figure 2.

Figure 2: A Phased Approach to Maximize the Impact of AI in the SDLC

A Phased Approach To Maximize The Impact of AI in The SDLC



Source: Gartner
843405

Gartner

Cautions

- **Avoid the pitfalls of increased security risks** by reinforcing the need for basic security hygiene and implementing the requisite level of human oversight (based on the application's business criticality, acceptable risk threshold, and architectural complexity).
- **Don't limit productivity benefits by only focusing on coding efficiency**, as developers typically spend more time on noncoding tasks like planning, design, architecture, and testing. Doing so runs the risk of "shifting" versus "saving" effort. For example, you might offset the time saved during development managing a bigger backlog for code reviews.
- **Mitigate the risk of skills erosion** through continuous development of core engineering and critical thinking abilities. AI dependency erodes independent judgment and hampers productivity whenever tools fail or are unavailable.

Adopt a Three-Phased Approach to Maximize the Impact of AI on the SDLC

Phase 1: Resolve Constraints by Mapping the Software Delivery Value Stream

Step 1: Identify and Resolve Systemic Bottlenecks in the SDLC by Judiciously Using AI and Non-AI Automation Techniques

To identify and resolve systemic bottlenecks in the SDLC, software engineering leaders must map the complete software delivery value stream, prioritizing constraints that obstruct the flow of work both across phases (such as handoffs between teams) and within individual phases (such as inner development loops).

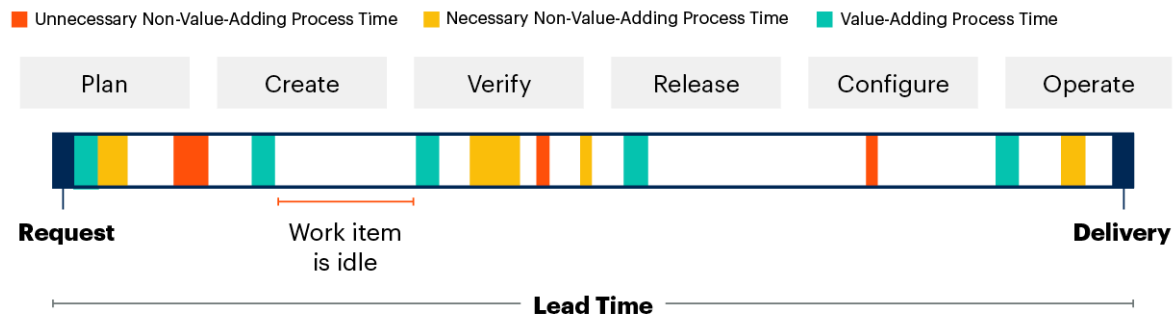
AI is not the solution to resolving every productivity blocker.

It is essential to recognize that AI is not the solution to resolving every productivity blocker. Traditional (non-AI) automation practices, such as continuous integration (with unit test automation), static code analysis, and automated production deployments, should be considered a basic prerequisite before introducing AI for more advanced use cases. A piecemeal approach to using AI for ad hoc use cases, particularly when systemic inefficiencies exist, can amplify those inefficiencies rather than addressing them.

For example, CBRE, a commercial real estate company, determines high-impact use cases of AI by identifying productivity pain points and assessing where AI is the best-fit solution to address them. For example, while monitoring errors was time-consuming, CBRE found that effectively leveraging an existing **data observability** tool was more viable than investing in new AI solutions. See [Case Study: Identify AI Use Cases That Maximize Developer Productivity](#).

The overall goal is to enhance the efficiency of delivery workflows by iteratively identifying the greatest constraint and resolving it, leveraging the respective strengths of both AI and established automation practices. Therefore, focusing AI narrowly on tasks like code generation alone fails to deliver substantial, system-level gains, because the bottleneck shifts downstream to areas such as code review, which requires additional automation tools and practices. See Figure 3.

Figure 3: Identify Constraints as Work Items Move Through the SDLC

Identify Constraints as Work Items Move Through the SDLC

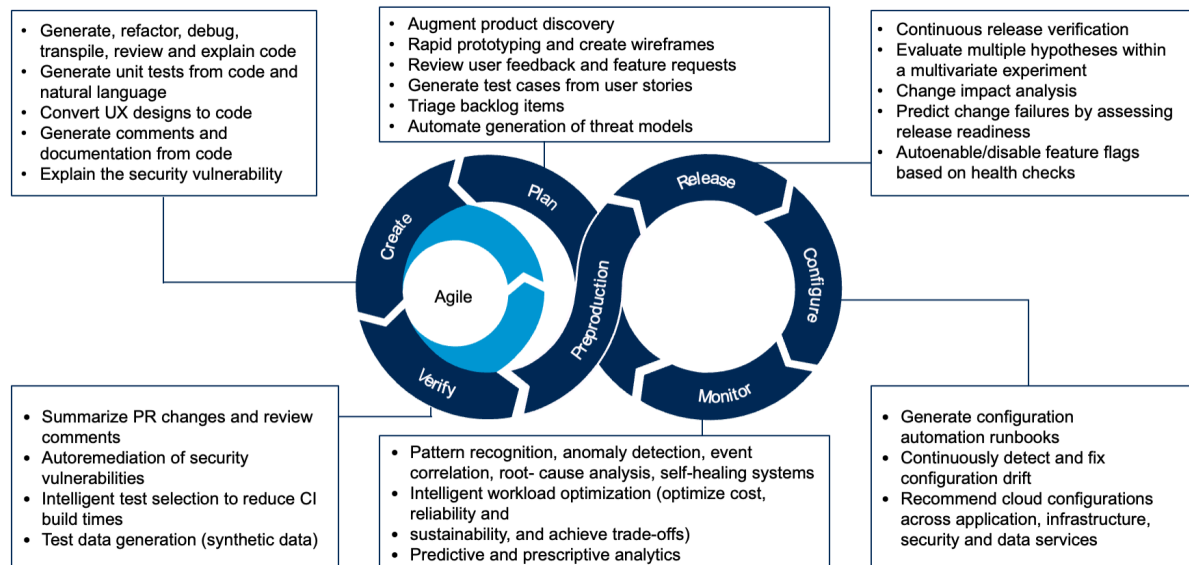
Source: Gartner
801906_C

Gartner

The 2025 Gartner AI in Software Engineering Survey shows that AI use cases are still concentrated in the inner loop of the SDLC. More than 50% of respondents use AI for writing and understanding code, debugging, generating documents, and writing test cases. Less than 50% of organizations utilize AI in both upstream use cases, such as gathering requirements, ideation, design, and generating user stories, and in downstream use cases, including code reviews, deployment, and monitoring. Refer to Figure 4 for examples of AI-augmented use cases throughout the SDLC.

Figure 4: Examples of AI-Augmented Use Cases Across the SDLC

Examples of AI-Augmented Use Cases Across The SDLC



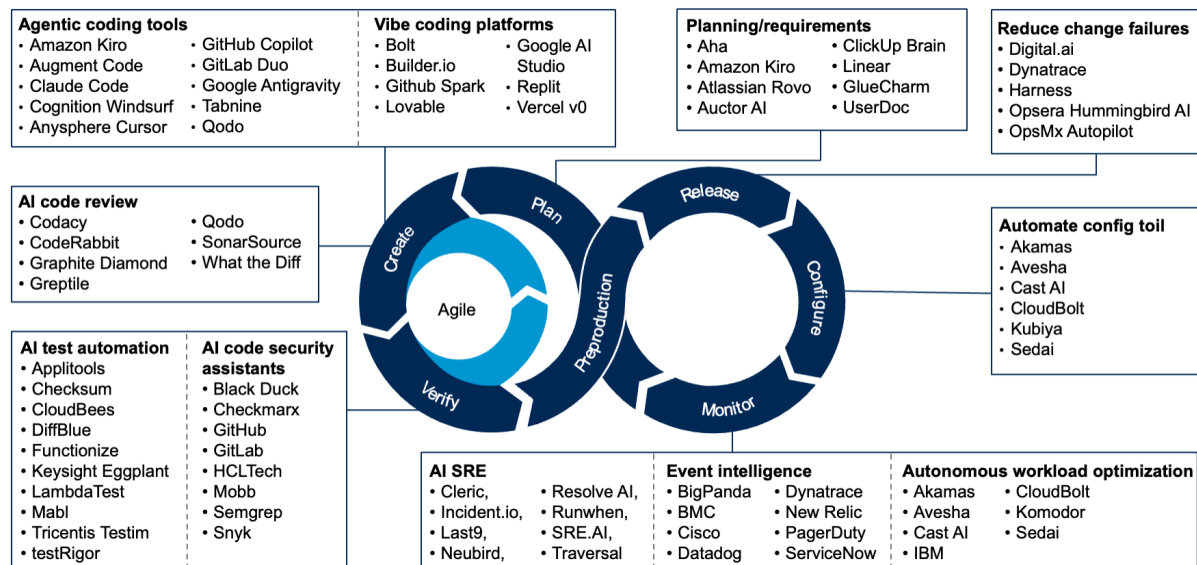
Source: Gartner
843405

Gartner

Figure 5 provides an illustrative example of AI tools across SDLC use cases.

Figure 5: Examples of AI Tools Across SDLC Use Cases

Examples of AI Tools Across SDLC Use Cases



Source: Gartner
843405

Gartner

Step 2: Discover Pain Points and Improve the Experience for All Roles, Not Just Developers

Maximizing productivity gains from AI requires moving beyond a narrow focus on developers and augmenting the workflows of all roles involved in product delivery. This is crucial because optimizing software delivery performance involves multiple roles like product owners, UX designers, platform engineers, security engineers, quality engineers, AI engineers, and data engineers.

AI can transform the experience of people in upstream planning phases by serving as an ideation partner for roles such as product owners and UX designers, enabling them to convert text prompts or visual sketches into prototypes and supporting better and faster decisions. This augmentation reduces cognitive load and repetitive work, facilitating a state of flow and enabling people to maximize their creative potential. See [Use AI-Augmented UX Tools to Empower, But Not Replace, Your Designers](#).

A recent case study of Vizient's human-centric approach emphasizes creating "empathy maps." The user empathy maps capture the distinct needs and desires for each role and push people to confront crucial questions about how AI impacts their work, identity, and life, specifically asking:

- "How will AI impact my job?"
- "How will AI impact my responsibility or ownership?"
- "How will AI impact my work schedule?"

Since capturing value from GenAI requires behavioral change, it's critical to understand how employees view their work. Software engineering leaders at Vizient utilize these questions to gain insight into how employees perceive their work and identity. See [Craft a Human-Centric Generative AI Strategy: Lessons from Vizient](#).

Step 3: Establish Internal Developer Platforms for Consistency and Scale

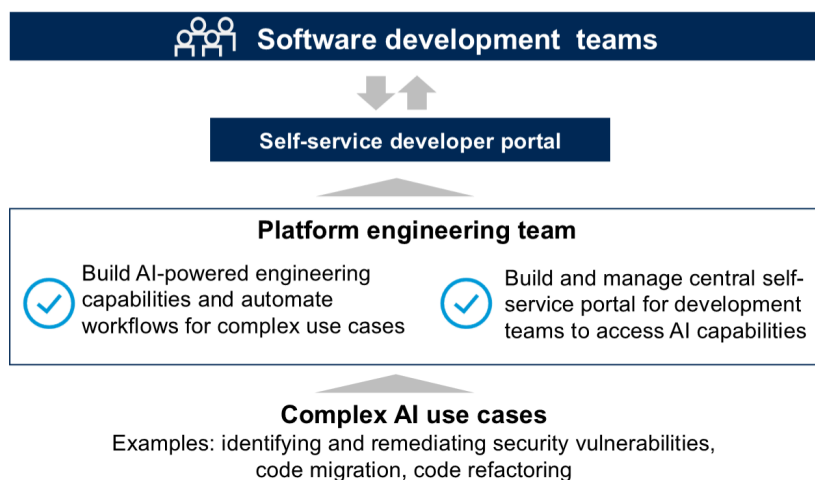
Establishing internal developer platforms via platform engineering is a foundational practice necessary to achieve consistency and scale when integrating AI capabilities across the SDLC. Internal developer platforms enable "paved roads" that represent a deliberately designed path. By integrating a common, standardized set of AI tools into these paths, a platform engineering team can help scale AI adoption and drive consistency across the full SDLC.

The Southern Company case study illustrates why scaling AI-augmented workflows requires platform engineering. As development teams at Southern Company began using AI for security vulnerability remediation, they struggled because they lacked the expertise for effective prompt and **context engineering** and found it time-consuming to ensure AI outputs aligned with the organization’s coding standards. See [Case Study: Scaling AI-Augmented Software Engineering Requires Platform Engineering](#).

To overcome this challenge, Southern Company leveraged its internal developer platform as an abstraction layer for AI capabilities, making AI tools easy and self-service for product teams. By creating automated workflows, such as a “Code Transformation Service,” the platform team standardized complex tasks, thereby abstracting the need for developers to write intricate prompts or spend time understanding complex AI-generated outputs. This platform-led approach ensures consistent security practices, scalable adoption of frequently used tools, and frees developers to focus on delivering business-critical features. See Figure 6.

Figure 6: Platform Engineering’s Role in Delivering AI Capabilities

Platform Engineering’s Role in Delivering AI Capabilities



Source: Adapted from Southern Company
835295



Phase 2: Reimagine the SDLC With Asynchronous Workflows

Step 1: Parallelize Tasks Using Asynchronous Agentic Workflows

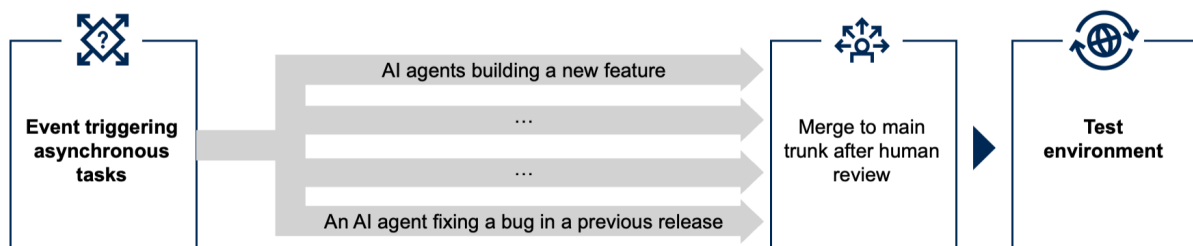
The rise of agentic coding tools and IDEs (e.g., Amazon Kiro, Cursor, Google Antigravity) supporting spec-driven development elevates the role of AI from being “code assistants” to “feature builders.” However, building features involves agents executing multistep tasks and is time-consuming. To avoid idle wait times, developers must run multiple tasks in an asynchronous manner, making parallelism a dire necessity.

For example, AI agents can asynchronously perform automated code reviews, autoremediate security vulnerabilities, fix bugs from previous product releases and refactor code to address “bad smells” in code. This parallel processing improves cycle time efficiency and effectively multiplies organizational capacity by handling maintenance tasks in the background while humans focus on high-value feature development.

However, the current technological maturity of agentic AI prevents complete consistency, accuracy, or predictability, making human-in-the-loop oversight mandatory to contain risk. The developer’s role evolves into that of a digital conductor who orchestrates the agents, clearly articulates intent while still reviewing and remaining accountable for agent-generated outcomes. See Figure 7.

Figure 7: Example of AI Agents Asynchronously Executing Tasks With a Human in the Loop

Example of AI Agents Asynchronously Executing Tasks With a Human in the Loop



Source: Gartner
843405

Gartner.

Step 2: Transform Workflows – Identify What Can Be Eliminated and What Stays the Same

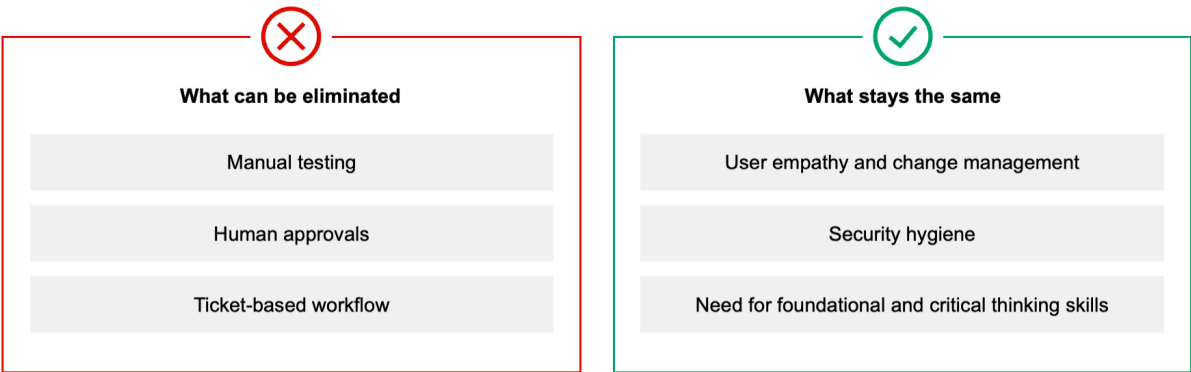
Maximizing the impact of agentic AI in the SDLC requires going beyond augmenting existing software development workflows and transforming the end-to-end delivery process. Software engineering leaders must identify opportunities for eliminating mundane, high-volume tasks rather than trying to improve their efficiency.

Examples of activities ripe for elimination include non-value-adding processes such as the drudgery of large-scale manual testing, raising tickets for provisioning development and test environments or steps that involve routine and repetitive human approvals. To identify such activities, start by asking (iteratively) why the process exists and look to eliminate or optimize the solution when the process indeed needs to exist.

Conversely, a few foundational human elements and processes must remain in place. For example, the need for user empathy and effective change management are still essential, requiring leaders to implement a human-centric strategy that adapts the pace of innovation to the pace at which users can absorb the change. See Figure 8.

Figure 8: Examples of Redundant Steps and Invariants in the SDLC

Examples of Redundant Steps and Invariants in the SDLC



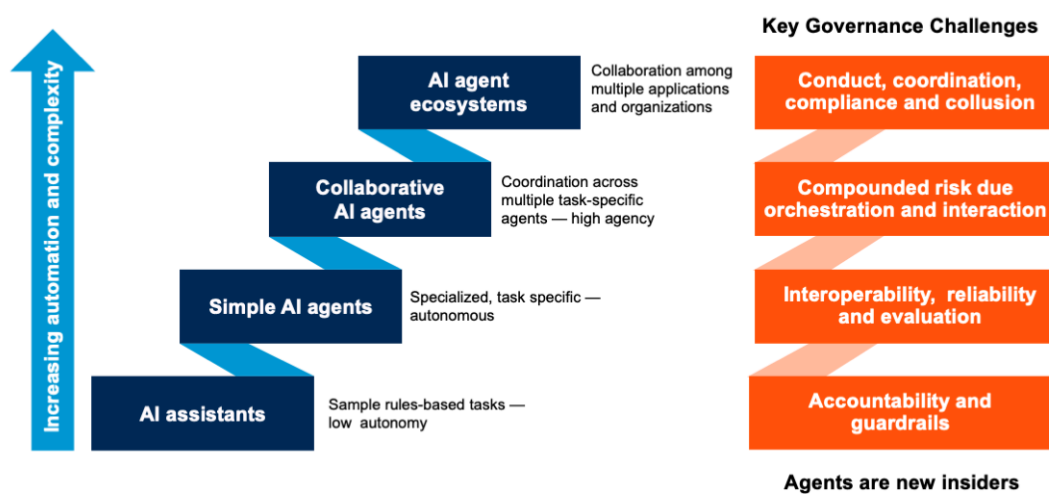
Source: Gartner
843405

Step 3: Enhance Internal Developer Platforms to Govern and Monitor AI SWE Agents

As engineering teams offload and delegate development tasks to AI agents, the increased autonomy and complexity of those agents necessitate the need for robust governance and mitigation of compounded risks. Instead of every team addressing governance issues on their own, platform engineering teams are uniquely positioned to strengthen internal developer platforms and “paved roads” (established in Phase 1) to provide a consistent approach for enforcing security guardrails, managing costs, reducing fragmentation and tackling interoperability issues. See Figure 9.

Figure 9: Levels of Autonomy and Related Governance Challenges

Levels of Autonomy and Related Governance Challenges



Source: Gartner
833878

Gartner

The internal developer platform becomes a scaling mechanism for setting and regularly updating guardrails and constraints for AI software engineering agents. These guardrails prevent the agent from calling unapproved tools and at the same time make it easier to use approved tools via catalogs and prebuilt integrations (e.g., model catalogs, MCP server catalogs, **AI engineering tools**).

Platform engineering teams can strengthen internal developer platforms to support AI agents in four ways:

- **Guardrails:** Ensures compliance via transparent data logging and protects systems by blocking invalid requests.

- **Credential management:** Centralize API key management and support secrets managers to enable secure access control and monitoring access paths.
- **Sandboxed environments:** Provides secure, isolated infrastructure for running AI agents and AI-generated code, enabling safe testing, evaluation, and deployment.
- **Cost management:** Ensure AI resources and inference costs are monitored, alerting teams to anomalous spending patterns.

Phase 3: Realize Zero-Friction SDLC With Autonomous Software Delivery

Step 1: Implement Autonomous Self-Improvement Loops and Address Pitfalls

Though still a big leap, autonomous software delivery shows two early signs of materializing:

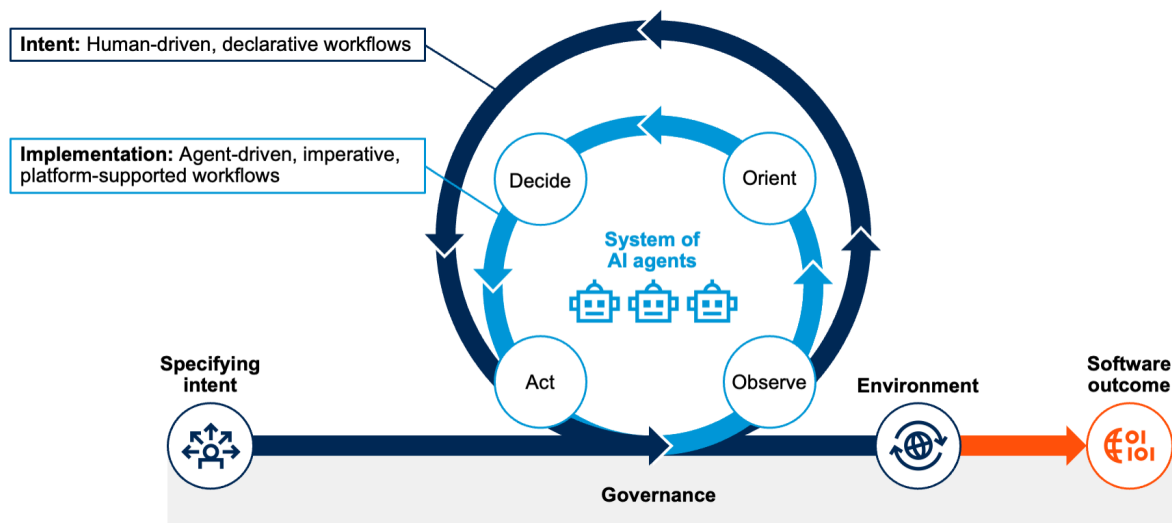
1. **Vibe coding platforms** that support a prompt-to-app workflow enable users to describe the desired functionality or interface of an application in natural language, and the underlying system handles everything else from architecture to code generation to deployment. The developer no longer interacts with source code, chooses technologies or manages environments. This gives developers uninterrupted focus on ideas and outcomes, not implementation. Examples of such tools, popularly called “vibe coding” tools include Lovable, Bolt, Vercel, Replit. See [Innovation Insight for Vibe Coding Platforms](#).
2. **Autonomous workload optimization** is paving a new era of autonomous IT operations by replacing static, human-defined automation rules with AI-driven systems that independently plan, decide, and act without continuous human intervention. This continuous, context-aware, autonomous approach to IT operations successfully tackles the challenge of performing multiobjective optimization — balancing the need to optimize costs, reliability, and sustainability simultaneously. See [Innovation Insight for Autonomous Workload Optimization](#).

While the above examples highlight very disparate use cases, they share a common thread. In both cases, the nature of human work shifts from being **imperative** (focusing on the “how”) to **declarative** (focusing on the “what”). This declarative approach is at the heart of future autonomous systems that rely on humans clearly articulating intent and AI agents translating the intent into testable, working software. We see a similar precedent in automation: the move from imperative to declarative modes, best demonstrated by GitOps.

Autonomous software delivery takes this idea a notch further by elevating the GitOps approach to the entire SDLC – where humans specify intent and a system of AI software engineering agents works in concert to translate that intent into deployable and testable software. This frictionless approach in going from intent to business outcomes paves the way for what Gartner calls **zero-friction SDLC**. For more details, see [Future of Software Engineering 2030: Zero-friction SDLC](#). Figure 10 shows a conceptual view of zero-friction SDLC.

Figure 10: Zero-Friction SDLC

Zero-Friction SDLC



Source: Gartner
843458

Gartner

Step 2: Expand Platform Support for Autonomous Delivery and Operations

To support a zero-friction SDLC and scale autonomous delivery, platform teams must prioritize three foundational capabilities in internal developer platforms:

- **Self-service AI tools and AI-powered capabilities:** Platform teams must provide self-service access to AI tools for software engineering via **internal developer portals**. These portals serve as a single entry point for development teams to discover and access AI capabilities and tools. For complex use cases, platform teams should create standardized, automated workflows and provide these as self-service capabilities to development teams. This abstraction layer simplifies the use of complex AI tools, such as those needed for autonomous workload optimization, autoremediation of security vulnerabilities or legacy code modernization.

- **Augmented FinOps:** Software engineering teams are not experts in cloud cost optimization, leading to either overprovisioning and spending more than needed, or underprovisioning and associated resource contention. This poses a challenge for autonomous software delivery since it requires a shift from a manual, reactive process to a proactive, real-time system for managing and optimizing cloud costs and resource usage.

Augmented FinOps is the use of AI for cloud financial management to analyze real-time data and provide predictive insights, accurate forecasting, anomaly detection, and intelligent, tailored recommendations for cost optimization. This helps intelligently automate repetitive tasks like identifying idle resources, rightsizing instances, enforcing tagging policies, and executing cost-saving actions. See [Hype Cycle for IT Operations, 2025](#).

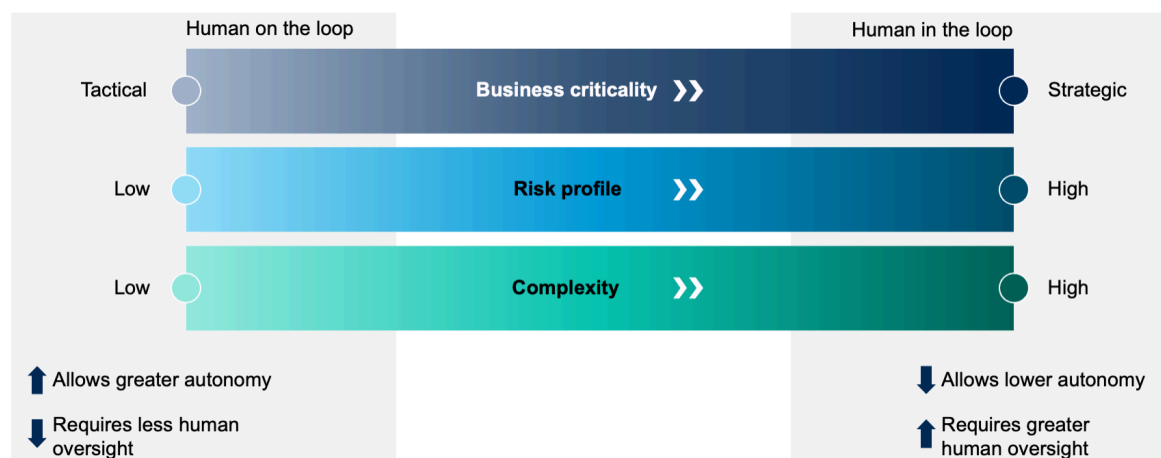
- **Augmented SRE:** Autonomous software delivery makes autonomous software reliability paramount. We cannot achieve autonomous software delivery if our safety net (operations and reliability) remains manual, reactive, and slow. Self-healing and self-correcting systems therefore become crucial to ensuring that the reliability of the system keeps pace with the speed of software delivery. Gartner predicts that by 2029, 60% of SREs will offload routine incident and postmortem tasks to AI agents, up from 5% in 2025. See [Innovation Insight for AI-augmented SRE](#) and [AI, Autonomy, and Architects: The Future of Site Reliability Engineering](#).

Step 3: Implement Appropriate Human Oversight for Autonomous Workflows Based on Business Criticality, Acceptable Risk, and Architectural Complexity

Autonomous software delivery requires carefully determining the necessary level of human oversight based on three key factors: business criticality, acceptable risk profile, and architectural complexity. These factors define a sliding scale that dictates whether tasks are suitable for full autonomy (human on the loop) or strict human supervision (human in the loop). Human in the loop requires explicit, continuous human approval before an agent executes decisions while human on the loop allows autonomous execution by the agent with humans supervising and stepping in only when exceptions, anomalies or alerts require action. See Figure 11.

Figure 11: Factors Determining the Level of Human Oversight

Factors Determining the Level of Human Oversight



(Example use cases: Internal DevOps tools, business enablement apps)

Source: Gartner
843405

(Example use cases: Customer-facing apps, apps offering core competitive advantage)

Gartner

Guidelines for Determining and Implementing Appropriate Level of Human Oversight

Tasks that rank low in criticality, risk, and complexity are prime candidates for full or near-full autonomy. These are routine, repetitive activities making it harder to justify business impact. Sample use cases for human on the loop include:

- Internal business enablement applications
- Minor bug fixes, dependency updates, and configuration changes
- Refactoring entire codebases to fix code smells or replace deprecated functions/APIs
- Generating documentation for code and architecture
- Generating unit test suites, test cases, or test data
- Autonomous workload optimization (e.g., predicting demand for autoscaling, rightsizing recommendations, autoconfiguration based on traffic patterns)

Tasks that are high in criticality, risk, or architectural complexity require substantial human involvement and judgment. Sample use cases for human in the loop include:

- Strategic activities such as product management, feature prioritization, understanding complex user needs, and software architecture
- Building business-critical software that constitutes core intellectual property
- Handling code related to financial calculations, sensitive data, or authorization logic
- Designing new features or architectural changes that impact systems of record or shared modules
- Verifying AI-generated solutions for business-critical or safety-critical use cases

Success Measures

Measuring success demands that organizations look beyond traditional output-based productivity metrics (with a focus on maximizing delivery efficiency) with outcome-based creativity metrics (with a focus on capturing value from ideation, experimentation, and strategic impact). This ensures the time saved by AI is reinvested into building novel, differentiated solutions.

In early stages of the SDLC, AI tools should serve as ideation partners to accelerate experimentation and drive breakthrough product innovation, especially in upstream creative activities such as requirements gathering, user story creation, and ideation, which yield more transformative impact than mere productivity improvements. See [Future of Software Engineering: A New Approach to Developer Productivity](#).

The approach taken by Vizient, a healthcare services company, demonstrates that successfully capturing value requires leaders to deliberately “co-design” the human-AI relationship. It empowers employees to experiment with personalized, role-based metrics, rather than forcing productivity measures that may not reflect actual value creation. See [Craft a Human-Centric Generative AI Strategy: Lessons from Vizient](#). Figure 12 shows how leaders can tailor metrics to specific roles rather than creating one-size-fits-all metrics.

Figure 12: Early Results From Enabling Employees to Co-Create the Future of Their Work^a

Early Results From Enabling Employees to Co-Create the Future of Their Work^a

 Sam Software Engineer	1. Reduced Effort • Reduced time on problem solving by 35% • Reduced code generation time by 40% 2. Increased Speed • Story completion – from 1 day to ¼ day • 80% faster code knowledge transfer 3. Improved Quality • 80% increase in code clarity • 15% improvement in code quality 4. Estimated Increased Adoption • 100% of developers trained on the tools • 70% of developers using the tools daily Employee Outcome Examples • Reduced roadblock fatigue • Increased work fulfilment • Development of new skills
 Becky Business Analyst	1. Reduced Effort • Requirements gathering rework fell by 100% • Documentation effort reduced by 75% 2. Increased Speed • Reduced user story generation time by 50% • Backlog generation faster by 3x 3. Improved Quality Improved quality in breadth of acceptance criteria for all user stories 4. Estimated Increased Adoption • 100% of business analysts trained on tools • 70% of business analysts use tools daily Employee Outcome Examples • More strategic thinking opportunities • Reduced time on unfulfilling tasks • Reduced silos in business and tech lingo

Source: Adapted from Vizient

^a Vizient used tools such as GitHub, Microsoft Copilot, and VizChat (Vizient's own, secure version of ChatGPT).

811249_C

vizient.

Gartner.

Evidence

2025 Gartner AI in Software Engineering Survey. This study was conducted to explore the adoption of AI within software engineering functions, focusing on two key areas: the use of AI tools (e.g., AI code assistants, AI code agents) throughout the software engineering life cycle (SDLC); and the development of AI-powered solutions (or AI engineering) within software engineering functions, along with their contribution to business outcomes. The research was conducted online from 29 April through 25 June 2025 among 299 respondents from North America (n = 150), EMEA (n = 104) and Asia/Pacific (n = 45). Quotas were established for company sizes and for industries to ensure a good representation across the sample. Organizations were required to be either piloting or using AI tools in SDLC for less than four years, and either piloting or having built AI solutions in their software engineering functions. Respondents included both leaders and individual contributors from software engineering functions, each with at least one year of tenure at their current organization. All respondents were involved in decision making or directly engaged in using AI tools or building AI solutions within their software engineering functions. Disclaimer: The results of this survey do not represent global findings or the market as a whole, but reflect the sentiments of the respondents and companies surveyed.

Document Revision History

[How Platform Engineering Teams Can Augment DevOps With AI - 8 January 2024](#)

Recommended by the Authors

Some documents may not be available as part of your current Gartner subscription.

[Expand the Use of AI Tools Across Software Development to Maximize Productivity](#)

[How to Capture AI-Driven Productivity Gains Across the SDLC](#)

[Innovation Insight for AI-Native Software Engineering](#)

[Software Engineering Foundations for the AI-Native Era](#)

[Innovation Insight for AI Software Engineering Agents](#)

© 2026 Gartner, Inc. and/or its affiliates. All rights reserved. Gartner is a registered trademark of Gartner, Inc. and its affiliates. This publication may not be reproduced or distributed in any form without Gartner's prior written permission. It consists of the opinions of Gartner's Business and Technology Insights Organization, which should not be construed as statements of fact. While the information contained in this publication has been obtained from sources believed to be reliable, Gartner disclaims all warranties as to the accuracy, completeness or adequacy of such information. Although Gartner insights may address legal and financial issues, Gartner does not provide legal or investment advice and its insights should not be construed or used as such. Your access and use of this publication are governed by [Gartner's Usage Policy](#). Gartner prides itself on its reputation for independence and objectivity. Its insights is produced independently by its Business and Technology Insights Organization without input or influence from any third party. For further information, see "[Guiding Principles on Independence and Objectivity](#)." Gartner insights may not be used as input into or for the training or development of generative artificial intelligence, machine learning, algorithms, software, or related technologies.